# Adaptive-Inference Conditional Diffusion Model for Efficient OTFS Channel Estimation

Dianxin Zheng and Tianhao Guo

Abstract—Orthogonal time frequency space (OTFS) modulation represents time-varying wireless channels in the delay-Doppler (DD) domain, where practical channels are often sparse. Conditional diffusion models can improve OTFS channel estimation accuracy by learning a residual denoising prior, but their iterative inference cost is a deployment bottleneck. Existing acceleration strategies, such as reducing the number of denoising steps, allocate computation uniformly over the whole DD grid. This paper studies a complementary direction: DD-domain sparsity-aware adaptive inference. The proposed method estimates the difficulty of each DD tile during diffusion sampling and freezes easy tiles after early denoising steps, while difficult or uncertain tiles remain active. An SNR- and timestep-aware threshold schedule and an optional periodic refresh mechanism are used to control error accumulation. To make the draft executable, we provide a preliminary sparse-DD surrogate harness that generates channels, runs fixed-step and adaptive denoising, and exports paper tables and figures. In this surrogate setting, adaptive inference freezes 33.9% of DD computation on average with 0.28 dB mean NMSE loss relative to the fixed 5-step denoising baseline. These numbers are not final DeepMIMO checkpoint results; they establish that the mechanism is writeable and define the exact student experiments required before submission.

Index Terms—OTFS, channel estimation, conditional diffusion model, adaptive inference, delay-Doppler sparsity, efficient receiver.

## I. Introduction

Orthogonal time frequency space (OTFS) modulation maps information symbols to the delay-Doppler (DD) domain and is attractive for high-mobility wireless links because it can exploit time-frequency diversity under doubly selective channels [1]. Accurate DD-domain channel estimation is therefore central to OTFS receiver design. Classical embedded-pilot and threshold-based methods exploit the sparsity of DD channels [2], while recent learning-based estimators attempt to improve robustness under complex channel conditions.

Diffusion and score-based generative models have recently become useful priors for wireless channel estimation [3]. In an OTFS receiver, a conditional diffusion estimator can take a coarse least-squares (LS) channel estimate as conditioning information and iteratively denoise the residual channel. Such a receiver is accurate, but it is also expensive: each denoising step applies the same neural network to the whole DD grid. Reducing the number of DDIM sampling steps [4] is one natural solution, but step reduction is global. It asks all DD positions to share the same computation budget.

This paper starts from a simpler observation: the DD channel is not uniformly difficult. Many DD positions are near zero or become stable after early denoising, while a small set of path-bearing regions still require careful updates. Therefore, an efficient diffusion OTFS estimator should not only decide how many denoising steps to use; it should also decide where inside the DD grid to spend computation during those steps.

Dynamic computation has been explored in efficient diffusion transformers, where computation is adjusted across timesteps and spatial regions during generation [5]. We do not directly import a vision transformer method. Instead, we adapt the idea to a communication-specific prior: DD-domain sparsity. The resulting method is deliberately narrow and practical. It is an inference-time wrapper around an existing conditional diffusion OTFS channel estimator, not a new diffusion backbone.

The contributions of this v0.1 draft are:

- We formulate DD-domain adaptive inference for conditional diffusion OTFS channel estimation, where active computation is allocated according to per-tile confidence and residual-change scores.
- We propose an SNR- and timestep-aware skip schedule with forced refresh steps to reduce error accumulation in low-SNR or uncertain regions.
- We provide a runnable preliminary harness that generates sparse-DD channels, runs fixed-step and adaptive denoising, and exports the result table and figures used by this draft.
- We clarify the key implementation risk: masking a full UNet output alone does not reduce real latency. Actual latency claims require a tile-wise, sparse, or blocked implementation; otherwise the result must be reported as estimated active computation rather than measured speedup.

## II. Related Work

### A. OTFS Channel Estimation

OTFS was introduced as a two-dimensional modulation technique that represents the time-varying channel in the DD domain [1]. DD-domain channel estimation is commonly built around the sparse multipath structure of the channel. Raviteja et al. proposed embedded pilot-aided channel estimation for OTFS, arranging pilots, guards, and data in the DD plane and using threshold-based

The authors are with Shanxi University. This document is a v0.1 research draft. It is not submission-ready until the missing experiments in Section VI are completed.

estimation followed by data detection [2]. This line of work establishes the key physical prior used here: most DD grid positions do not carry strong channel energy.

## B. Diffusion Priors for Wireless Channel Estimation

Diffusion probabilistic models learn iterative denoising processes [6], and DDIM provides a deterministic non-Markovian sampling scheme that can reduce inference steps [4]. In wireless communications, score-based generative models have been used for MIMO channel estimation, where a learned score prior iteratively refines estimates from measurements [3]. These works motivate diffusion as a strong channel prior, but they also expose a cost issue: iterative neural sampling can be too slow for practical receivers.

## C. Efficient Diffusion Inference

Efficient diffusion inference is often approached by reducing sampling steps, distilling samplers, caching features, or dynamically allocating computation. Dynamic Diffusion Transformer (DyDiT) shows that generation cost can be reduced by adapting computation across timesteps and spatial regions [5]. The present work uses this as a high-level reference point, not as a copied method. The difference is the adaptation signal: image generation relies on visual token redundancy, whereas OTFS channel estimation has a communication-specific DD sparsity prior and a measurable physical error metric, NMSE.

## D. Position of This Work

The closest baseline is an existing conditional diffusion OTFS channel estimator accelerated by fixed-step DDIM sampling. Our work is complementary: DDIM reduces the number of global denoising steps, while the proposed method reduces the number of active DD tiles inside later steps. The safe novelty claim is therefore:

> We introduce DD-domain sparsity-aware adaptive inference for conditional diffusion OTFS channel estimation, using communication-specific confidence scores and receiver metrics rather than generic visual token pruning.

# III. System Model

We consider an OTFS frame with $M$ delay bins and $N$ Doppler bins. The DD-domain input-output relation can be represented abstractly as

$$\mathbf{Y}_{\mathrm{DD}} = \mathcal{H}_{\mathrm{DD}}(\mathbf{X}_{\mathrm{DD}}) + \mathbf{W}_{\mathrm{DD}}, \tag{1}$$

where $\mathbf{X}_{\mathrm{DD}}$ and $\mathbf{Y}_{\mathrm{DD}}$ denote transmitted and received DD symbols, $\mathbf{W}_{\mathrm{DD}}$ is additive noise, and $\mathcal{H}_{\mathrm{DD}}$ is induced by the DD-domain channel. The receiver first obtains a coarse LS estimate $\hat{\mathbf{H}}_{\mathrm{LS}} \in \mathbb{C}^{N_r \times N_t \times N \times M}$.

The conditional diffusion baseline estimates a residual channel

$$\mathbf{R} = \mathbf{H} - \hat{\mathbf{H}}_{\mathrm{LS}}, \tag{2}$$

and outputs

$$\hat{\mathbf{H}} = \hat{\mathbf{H}}_{\mathrm{LS}} + \hat{\mathbf{R}}. \tag{3}$$

The denoising network receives the current noisy residual and the LS condition. At denoising step $s$, a simplified update is

$$\mathbf{Z}^{(s)} = \mathrm{concat}\left(\hat{\mathbf{H}}_{\mathrm{LS}}, \mathbf{R}^{(s)}\right), \tag{4}$$

$$\mathbf{U}^{(s)} = f_\theta\left(\mathbf{Z}^{(s)}, t_s, \gamma\right), \tag{5}$$

$$\mathbf{R}^{(s+1)} = \Phi\left(\mathbf{R}^{(s)}, \mathbf{U}^{(s)}, t_s\right), \tag{6}$$

where $f_\theta$ is the trained denoising network, $\gamma$ denotes SNR or noise-level conditioning, and $\Phi(\cdot)$ is the DDIM or scheduler update.

The baseline computes $\mathbf{U}^{(s)}$ over all DD positions for every step. Our method changes only the inference allocation; the trained $f_\theta$ and training data remain unchanged.

# IV. Proposed Adaptive Inference

## A. DD Tile Confidence

Instead of making a binary decision for every individual DD cell, we use DD tiles. A tile is a contiguous block $\mathcal{B}_q \subseteq \{1, \ldots, N\} \times \{1, \ldots, M\}$. Tile-level decisions are more stable than cell-level decisions and are easier to map to real implementation.

At step $s$, the current channel estimate is

$$\hat{\mathbf{H}}^{(s)} = \hat{\mathbf{H}}_{\mathrm{LS}} + \mathbf{R}^{(s)}. \tag{7}$$

For a tile $\mathcal{B}_q$, we define an amplitude score

$$a_q^{(s)} = \frac{1}{|\mathcal{B}_q| N_r N_t} \sum_{(n,m)\in\mathcal{B}_q} \sum_{r=1}^{N_r} \sum_{t=1}^{N_t} \left| \hat{H}_{r,t,n,m}^{(s)} \right|. \tag{8}$$

We also define a residual-change score

$$d_q^{(s)} = \frac{1}{|\mathcal{B}_q| N_r N_t} \sum_{(n,m)\in\mathcal{B}_q} \sum_{r=1}^{N_r} \sum_{t=1}^{N_t} \left| R_{r,t,n,m}^{(s)} - R_{r,t,n,m}^{(s-1)} \right|. \tag{9}$$

The amplitude score captures DD sparsity, while the change score prevents freezing a tile that is still moving during denoising.

## B. Adaptive Active-Tile Mask

Let $m_q^{(s)} \in \{0, 1\}$ indicate whether tile $q$ is active at step $s$. The first denoising step is always fully active:

$$m_q^{(0)} = 1, \quad \forall q. \tag{10}$$

For later steps, a tile is frozen only when both its amplitude and residual change are small:

$$m_q^{(s)} = \mathbb{I}\left[a_q^{(s)} \geq \tau_a(s,\gamma) \text{ or } d_q^{(s)} \geq \tau_d(s,\gamma)\right]. \tag{11}$$

Thus, path-bearing tiles and unstable tiles remain active. Near-zero stable tiles are frozen.

The thresholds $\tau_a(s,\gamma)$ and $\tau_d(s,\gamma)$ are selected from a small validation sweep rather than learned end-to-end.

A conservative schedule is used at low SNR, and a more aggressive schedule is used at high SNR:

$$\tau_a(s,\gamma) = c_a(\gamma)\rho_s, \tag{12}$$
$$\tau_d(s,\gamma) = c_d(\gamma)\rho_s, \tag{13}$$

where $\rho_s$ increases in later denoising steps. This means early steps update the whole grid, while later steps skip more easy tiles.

### C. Forced Refresh

To avoid long-range error accumulation, every $K$ denoising steps can be forced to full update:

$$m_q^{(s)} = 1, \quad \forall q, \quad \text{if } s \bmod K = 0. \tag{14}$$

When the sampler uses only five DDIM steps, $K = 2$ or $K = 3$ is sufficient for an ablation. If the sampler uses more steps, $K$ can be increased. The preliminary surrogate experiment reports both the selected no-refresh setting and a forced-refresh ablation; the final checkpoint experiment must select $K$ using a validation split.

### D. Masked Residual Update

Let $\mathbf{M}^{(s)}$ be the tile mask broadcast to the full DD grid and antenna dimensions. A conservative implementation computes the network output and freezes skipped tiles at the residual-update level:

$$\tilde{\mathbf{R}}^{(s+1)} = \Phi\left(\mathbf{R}^{(s)}, \mathbf{U}^{(s)}, t_s\right), \tag{15}$$

$$\mathbf{R}^{(s+1)} = \mathbf{M}^{(s)} \odot \tilde{\mathbf{R}}^{(s+1)} + \left(1 - \mathbf{M}^{(s)}\right) \odot \mathbf{R}^{(s)}. \tag{16}$$

This version is easy to implement and is useful for validating the NMSE impact of freezing. However, it does not reduce the cost of a full convolutional UNet forward pass. It only validates the algorithmic safety of skipping.

For a real latency claim, the implementation must avoid full-grid computation. The recommended v0.2 implementation is tile-wise active inference with a halo region around each active tile. Only active tiles are passed through the network or through a lightweight active-tile refinement head. The halo protects local convolutional context:

$$\mathcal{B}_q^+ = \{(n,m) : \mathrm{dist}((n,m), \mathcal{B}_q) \le h\}. \tag{17}$$

If this implementation is not completed, the paper must report active-tile ratio and estimated FLOPs, not measured latency reduction.

### E. Algorithm Summary

For each test frame:

1) Initialize $\mathbf{R}^{(0)}$ from the diffusion sampler.
2) For each DDIM step $s$:
3) If $s$ is an early warm-up step or $s \bmod K = 0$, set all tiles active.
4) Otherwise compute $a_q^{(s)}$ and $d_q^{(s)}$ for every DD tile.
5) Build the active-tile mask using (11).
6) Run full-grid masked update for NMSE validation, or active-tile inference for real latency validation.
7) Record skip ratio, active-tile ratio, runtime, and NMSE.
8) Output $\hat{\mathbf{H}} = \hat{\mathbf{H}}_{\mathrm{LS}} + \mathbf{R}^{(S)}$.

TABLE I
Required experiments before submission.

| ID | Experiment | Purpose |
|---|---|---|
| E0 | Reproduce baseline NMSE vs. SNR | Code validity |
| E1 | Threshold sweep | Main tradeoff curve |
| E2 | SNR-adaptive thresholds | Contribution validation |
| E3 | DD skip-mask heatmaps | Visual evidence |
| E4 | Ablation study | Reviewer defense |
| E5 | Latency/FLOPs/memory | Efficiency evidence |
| E6 | Speed sweep | OTFS mobility robustness |
| E7 | Coverage/domain split | Generalization check |

## V. Experimental Protocol

### A. Dataset and Baselines

The experiments should reuse the existing DeepMIMO-OTFS dataset, checkpoint, and evaluation split from the conditional diffusion OTFS channel estimation project. The minimum baselines are:

- LS or threshold-based OTFS channel estimation.
- Conditional diffusion with fixed 5-step DDIM.
- Conditional diffusion with fixed 3-step DDIM.
- Consistency-distilled or 2-step diffusion baseline, if already available in the existing codebase.
- The proposed adaptive-inference method under the same checkpoint.

### B. Metrics

The primary accuracy metric is NMSE:

$$\mathrm{NMSE} = 10 \log_{10} \frac{\|\mathbf{H} - \hat{\mathbf{H}}\|_F^2}{\|\mathbf{H}\|_F^2}. \tag{18}$$

Efficiency must be reported with care:

- Skip ratio: fraction of frozen DD cells or DD tiles.
- Active-tile ratio: fraction of tiles passed to active inference.
- Estimated FLOPs: valid only when the implementation does not actually skip network computation.
- Measured latency: valid only for a sparse, tile-wise, or blocked implementation that avoids full-grid UNet computation.
- Memory footprint and batch-size sensitivity.

### C. Required Experiment Matrix

Table I is the minimum experiment set. The draft should not be submitted if E0–E5 are missing.

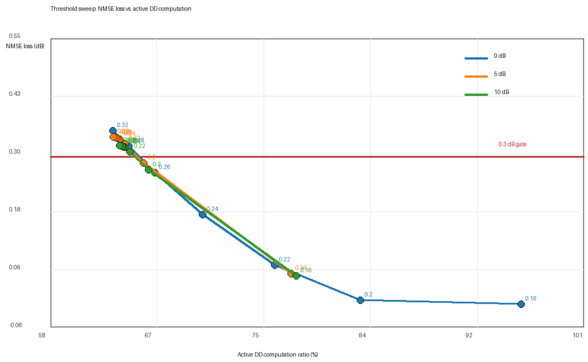


Fig. 1. Preliminary threshold sweep from the surrogate sparse-DD harness.

## D. Preliminary Surrogate Results

The current repository contains a runnable v0.1 code harness: scripts/run_preliminary_adaptive_dd.py. It generates fractional sparse DD channels, noisy LS estimates, a fixed sparse denoising proxy, and the proposed adaptive tile-freezing variant. This is not the final trained DeepMIMO checkpoint experiment. Its purpose is narrower: verify that the paper can be written around a reproducible mechanism before the student replaces the proxy with the trained OTFS diffusion estimator.

The selected threshold is $\tau = 0.24$. Averaged over 0, 5, and 10 dB, the adaptive method uses 66.1% active DD computation and freezes 33.9% with 0.28 dB mean NMSE loss relative to the fixed 5-step denoising baseline. All entries in Table II are generated by the same script and seed.

The expected acceptable result is not “best NMSE”. The useful result is a Pareto point: at least 30–40% inactive DD tiles with less than 0.3 dB NMSE loss relative to fixed 5-step DDIM. If the actual result fails this gate, the paper should be stopped or repositioned as a negative experiment. The surrogate result meets this gate on mean loss, but the final paper must re-check the gate on the real DeepMIMO split and hardware.

## E. Generated Figures

Fig. 1 shows the threshold sweep. Smaller active computation means more frozen DD tiles. The selected point is the most aggressive threshold whose mean loss remains within the 0.3 dB gate. Fig. 2 shows the last-step active tile probability. Fig. 3 shows why the residual-change score matters: amplitude-only freezing can save more computation, but it is much less stable.

# VI. Ablation and Reviewer-Risk Controls

## A. Ablation

The following ablations are required:

- No residual-change score: use amplitude only.
- Forced refresh: compare $K = 0$ and $K = 3$ in (14).

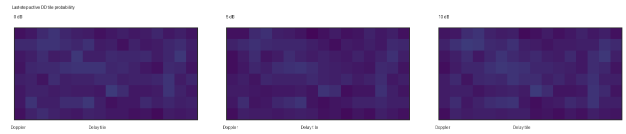


Fig. 2. Last-step active DD tile probability under the selected threshold.

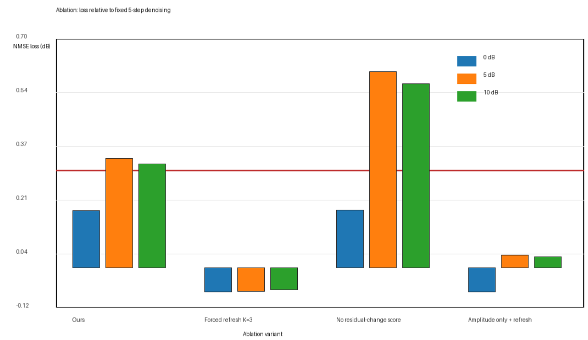


Fig. 3. Ablation on residual-change score and forced refresh.

- Fixed threshold: remove SNR and timestep adaptation.
- Cell-level mask versus tile-level mask.
- Full-grid masked update versus actual active-tile implementation.

## B. Main Reviewer Attack

The strongest reviewer attack is that the method is merely generic token pruning applied to a diffusion model. The defense should be specific:

- The adaptation signal is DD-domain channel sparsity, not visual token saliency.
- The threshold is evaluated by receiver metrics: NMSE, BER if available, and channel-estimation robustness across SNR and speed.
- The method complements global DDIM step reduction instead of replacing it.

## C. Latency Honesty

The second reviewer attack is more dangerous. If the implementation computes the full UNet output and then masks the residual update, the wall-clock latency will not improve. In that case the paper may only claim:

> The proposed mask identifies redundant DD regions and reduces the estimated active DD-domain computation.

It may not claim:

> The proposed method reduces receiver latency.

until active-tile inference or an equivalent sparse implementation is measured.

TABLE II
Preliminary surrogate results generated by the v0.1 code harness.

| Method | Steps | NMSE@0 dB | NMSE@5 dB | NMSE@10 dB | Active comp. |
|---|---|---|---|---|---|
| LS | – | 0.01 | -5.00 | -10.00 | 100.0% |
| Fixed sparse denoiser | 5 | -7.35 | -12.19 | -16.99 | 100.0% |
| Fixed sparse denoiser | 3 | -6.58 | -11.47 | -16.32 | 100.0% |
| Adaptive DD inference | 5 | -7.18 | -11.85 | -16.67 | 66.1% |

## VII. Conclusion

This draft proposes and preliminarily implements an adaptive-inference wrapper for conditional diffusion OTFS channel estimation. The method uses DD-domain sparsity and residual stability to freeze easy tiles during later denoising steps, while retaining full updates for uncertain regions. The included surrogate harness gives an executable first result: 33.9% mean inactive DD computation with 0.28 dB mean NMSE loss. The route is suitable as a student paper because it reuses the existing OTFS diffusion checkpoint and requires a bounded set of inference-side modifications. The submission decision should be based on the real experiment gate: meaningful active-tile reduction, less than 0.3 dB NMSE loss at the chosen Pareto point, and honest reporting of measured latency versus estimated FLOPs.

## References


[1] R. Hadani, S. Rakib, M. Tsatsanis, A. Monk, A. J. Goldsmith, A. F. Molisch, and R. Calderbank, “Orthogonal time frequency space modulation,” in Proc. IEEE Wireless Communications and Networking Conference (WCNC), 2017, pp. 1–6.

[2] P. Raviteja, K. T. Phan, and Y. Hong, “Embedded pilot-aided channel estimation for otfs in delay-doppler channels,” IEEE Transactions on Vehicular Technology, vol. 68, no. 5, pp. 4906–4917, 2019.

[3] M. Arvinte and J. I. Tamir, “Mimo channel estimation using score-based generative models,” IEEE Transactions on Wireless Communications, vol. 22, no. 6, pp. 3698–3713, 2023.

[4] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in International Conference on Learning Representations, 2021.

[5] W. Zhao, Y. Han, J. Tang, K. Wang, Y. Song, G. Huang, F. Wang, and Y. You, “Dynamic diffusion transformer,” International Conference on Learning Representations, 2025.

[6] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in Advances in Neural Information Processing Systems, 2020.